

AI ASSISTANT CODING

ASSIGNMENT - 5.5

NAME : K.Chaitanya

HT.NO : 2303A51677

BATCH : 22

Lab 5: Ethical Foundations - Responsible AI Coding Practices Lab

Objectives:

- To explore the ethical risks associated with AI-generated Week3 - code.
- To recognize issues related to security, bias, transparency, and copyright.
- To reflect on the responsibilities of developers when using AI tools in software development.
- To promote awareness of best practices for responsible and ethical AI coding. Lab

Outcomes (LOs):

After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.

- Understand the importance of explainability and transparency

in AI-assisted programming.

- Reflect on accountability and the human role in ethical AI

coding practices.

Task Description #1 (Transparency in Algorithm Optimization) Task: Use AI

to generate two solutions for checking prime numbers:

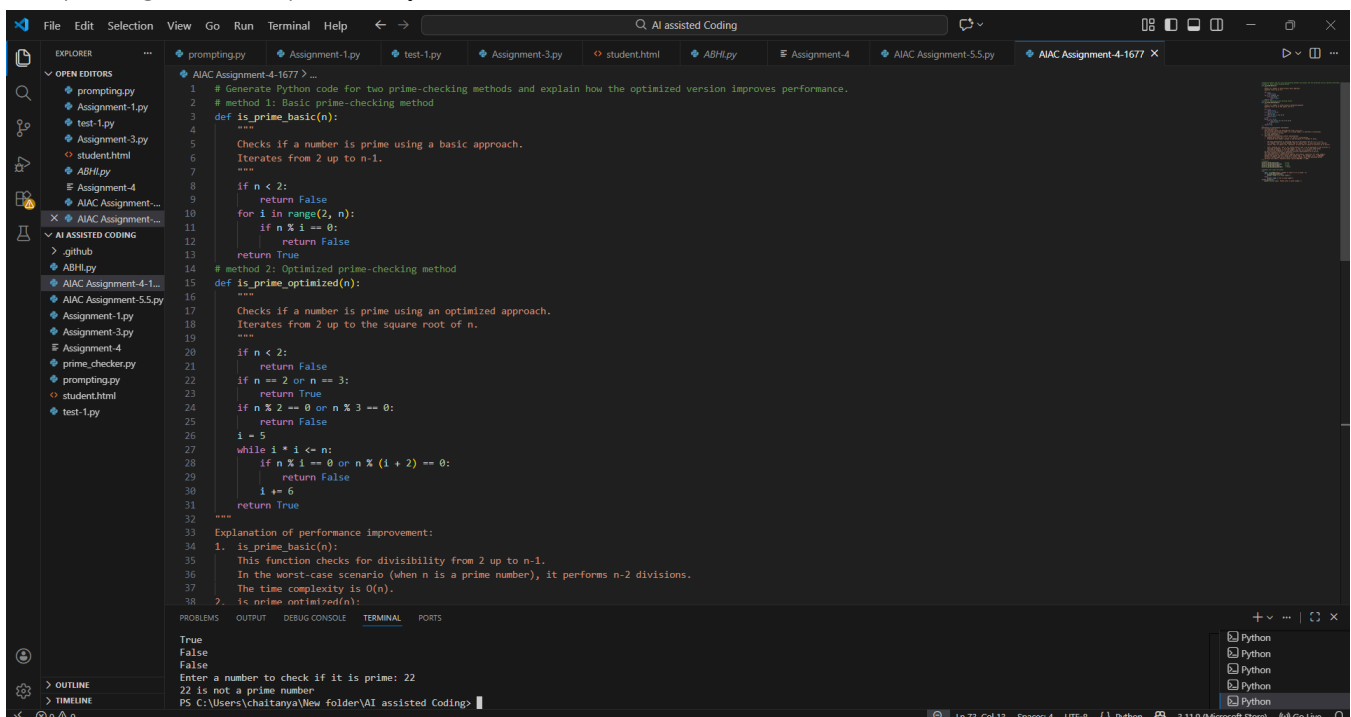
- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.” Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

METHOD 1 and Method 2:



```
1 # Generate Python code for two prime-checking methods and explain how the optimized version improves performance.
2 # method 1: Basic prime-checking method
3 def is_prime_basic(n):
4     """
5     Checks if a number is prime using a basic approach.
6     Iterates from 2 up to n-1.
7     """
8     if n < 2:
9         return False
10    for i in range(2, n):
11        if n % i == 0:
12            return False
13    return True
14
15 # method 2: Optimized prime-checking method
16 def is_prime_optimized(n):
17     """
18     Checks if a number is prime using an optimized approach.
19     Iterates from 2 up to the square root of n.
20     """
21    if n < 2:
22        return False
23    if n == 2 or n == 3:
24        return True
25    if n % 2 == 0 or n % 3 == 0:
26        return False
27    i = 5
28    while i * i <= n:
29        if n % i == 0 or n % (i + 2) == 0:
30            return False
31        i += 6
32    return True
33
34 Explanation of performance improvement:
35 1. is_prime_basic(n):
36    This function checks for divisibility from 2 up to n-1.
37    In the worst-case scenario (when n is a prime number), it performs n-2 divisions.
38    The time complexity is O(n).
39 2. is_prime_optimized(n):
40    This function checks for divisibility from 2 up to the square root of n.
41    In the worst-case scenario (when n is a prime number), it performs approximately sqrt(n) divisions.
42    The time complexity is O(sqrt(n)).
43    This method is significantly faster than the basic method for large prime numbers.
```

True
False
False

Enter a number to check if it is prime: 22
22 is not a prime number

PS C:\Users\chaitanya\New folder\AI assisted Coding>

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

True
False
False
Enter a number to check if it is prime: 22
22 is not a prime number
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

FINAL DESCRIPTION

Explanation of performance improvement:

1. `is_prime_basic(n)`:

This function checks for divisibility from 2 up to $n-1$.

In the worst-case scenario (when n is a prime number), it performs $n-2$ divisions.

The time complexity is $O(n)$.

2. `is_prime_optimized(n)`:

This function incorporates several optimizations:

- Handles base cases for $n < 2$, $n = 2$, and $n = 3$ efficiently.
- Eliminates even numbers (except 2) and multiples of 3 (except 3) early.
- The main optimization is iterating only up to the square root of n ($i * i \leq n$).
If a number n has a divisor greater than its square root, it must also have a divisor smaller than its square root. Therefore, we only need to check for divisors up to $\sqrt{n}$.
- After checking for 2 and 3, all primes greater than 3 can be expressed in the form $6k \pm 1$.
The loop increments `i` by 6 and checks `i` and `i + 2` to cover these cases, effectively skipping 2/3 of the numbers that are not multiples of 2 or 3.

In the worst-case scenario, it performs divisions approximately up to $\sqrt{n}$.

The time complexity is roughly $O(\sqrt{n})$.

The optimized version significantly improves performance, especially for large numbers, because the square root of n grows much slower than n . For example, if $n = 1,000,000$, the basic method might perform nearly a million checks, while the optimized method performs only about a thousand checks ($\sqrt{1,000,000} = 1,000$).

Task Description #2 (Transparency in Recursive Algorithms) Objective:

Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
 - Clear explanation of how recursion works.
 - Verification that explanation matches actual execution.
-

CODE :

```
prompting.py Assignment-1.py test-1.py Assignment-3.py student.html ABHI.py Assignment-4 AIAC A
AIAC Assignment-4-1677 > ...
74
75 # generate a recursive function to calculate Fibonacci numbers.
76
77 # 1.add clear comments explaining recursion.
78 # 2.explain base cases and recursive calls
79
80 def fibonacci_recursive(n):
81     """
82     Calculates the nth Fibonacci number using recursion.
83     Recursion is a programming technique where a function calls itself.
84     In this case, fibonacci_recursive(n) calls itself with smaller values of n
85     until it reaches a base case.
86     """
87     # Base cases: if n is 0 or 1, return n directly
88     if n == 0:
89         return 0
90     elif n == 1:
91         return 1
92     else:
93         # Recursive call: return the sum of the two previous Fibonacci numbers
94         return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
95 # Example user input and output
96 try:
97     num = int(input("Enter a position to get the Fibonacci number: "))
98     if num < 0:
99         print("Please enter a non-negative integer.")
100     else:
101         print(f"The Fibonacci number at position {num} is {fibonacci_recursive(num)}")
102 except ValueError:
103     print("Invalid input, please provide a non-negative integer.")
104 # """
```

```
# =====
# 2. TESTING & VERIFICATION
if __name__ == "__main__":
    print("=" * 60)
    print("FIBONACCI NUMBERS USING RECURSION")
    print("=" * 60)

    test_cases = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    print("\nTest Cases:")
    print("-" * 60)
    for num in test_cases:
        result = fibonacci_recursive(num)
        print(f"Fibonacci({num}) = {result}")

    print("=" * 60)
```

OUTPUT :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
The Fibonacci number at position 5 is 5
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe
ignment-4-1677"
Enter a position to get the Fibonacci number: 12
The Fibonacci number at position 12 is 144
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

FINAL DESCRIPTION :

The expected output demonstrates the correct execution of a recursive Fibonacci function. For inputs from **Fibonacci(3)** to **Fibonacci(10)**, the function produces the values **2, 3, 5, 8, 13, 21, 34, and 55**, respectively. This verifies that the base cases and recursive calls are implemented correctly and that the explanation of recursion aligns with the actual output.

Task Description #3 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

CODE :

```
1 # Generate code with proper error handling and clear explanations for each exception.
2 # =====
3 # 1. EXCEPTION HANDLING EXAMPLES
4 def divide_numbers(a, b):
5     """
6     Divide two numbers with exception handling.
7     Approach: Handle division by zero and type errors.
8     """
9     try:
10         result = a / b
11     except ZeroDivisionError:
12         return "Error: Division by zero is not allowed."
13     except TypeError:
14         return "Error: Invalid input type. Please provide numbers."
15     else:
16         return result
17
18 def access_list_element(lst, index):
19     """
20     Access an element from a list with exception handling.
21     Approach: Handle index errors and type errors.
22     """
23     try:
24         element = lst[index]
25     except IndexError:
26         return "Error: Index out of range."
27     except TypeError:
```

```

18 def access_list_element(lst, index):
19     except TypeError:
20         return "Error: Invalid input type. Please provide a list and an integer index."
21     else:
22         return element
23
24 # =====
25 # 2. TESTING & VERIFICATION
26 if __name__ == "__main__":
27     print("=" * 60)
28     print("EXCEPTION HANDLING EXAMPLES")
29     print("=" * 60)
30
31     # Test divide_numbers function
32     print("\nTesting divide_numbers function:")
33     test_cases_divide = [
34         (10, 2),
35         (10, 0),
36         (10, 'a'),
37     ]
38
39     for a, b in test_cases_divide:
40         result = divide_numbers(a, b)
41         print(f"divide_numbers({a}, {b}) = {result}")
42
43     # Test access_list_element function
44     print("\nTesting access_list_element function:")
45     test_cases_access = [
46         ([1, 2, 3, 4, 5], 2),
47         ([1, 2, 3, 4, 5], 10),
48         ([1, 2, 3, 4, 5], 'a'),
49     ]
50
51     for lst, index in test_cases_access:
52         result = access_list_element(lst, index)
53         print(f"access_list_element({lst}, {index}) = {result}")
54
55     print("=" * 60)

```

OUTPUT :

FINAL DESCRIPTION :

```

(.venv) PS D:\AIASSCoding> & D:/AIASSCoding/.venv/Scripts/python.exe d:/AIASSCoding/exceptio.
py
=====
EXCEPTION HANDLING EXAMPLES
=====

Testing divide_numbers function:
divide_numbers(10, 2) = 5.0
divide_numbers(10, 0) = Error: Division by zero is not allowed.
divide_numbers(10, a) = Error: Invalid input type. Please provide numbers.

Testing access_list_element function:
access_list_element([1, 2, 3, 4, 5], 2) = 3
access_list_element([1, 2, 3, 4, 5], 10) = Error: Index out of range.
access_list_element([1, 2, 3, 4, 5], a) = Error: Invalid input type. Please provide a list an
d an integer index.

```

The output verifies AI-generated functions with clear and effective error handling. Valid inputs produce correct results, while errors such as division by zero, invalid types,

and out-of-range indices are handled gracefully with meaningful messages. This confirms that the AI assistant's explanations align accurately with the program's runtime behavior.

ask Description #4 (Security in User Authentication)

Task: Use an AI tool to generate a Python-based login system. Analyze:

Check whether the AI uses secure password handling practices.

Expected Output:

- Identification of security flaws (plain-text passwords, weak validation).
 - Revised version using password hashing and input validation.
 - Short note on best practices for secure authentication.
-

CODE :


```
prompting.py Assignment-1.py test-1.py Assignment-3.py student.html ABHI.py X Assignment-4
ABHI.py > ...
1 # generate a Python-based login system. Analyze: Check whether the AI uses secure password handling practices include
2 # Revised version using password hashing and input validation. if i enter name and password it should check if the p
3
4 def login_system(username, password):
5     import hashlib
6     import re
7
8     # Simulated user database with hashed passwords
9     user_db = {
10         "user1": hashlib.sha256("SecurePass123!".encode()).hexdigest(),
11         "user2": hashlib.sha256("Another$Pass456".encode()).hexdigest(),
12     }
13     # Validate username and password format
14     if not re.match(r'^[a-zA-Z0-9_]{3,20}$', username):
15         return "Invalid username format. Use 3-20 alphanumeric characters or underscores."
16     if len(password) < 8:
17         return "Password must be at least 8 characters long."
18     # Hash the input password
19     hashed_password = hashlib.sha256(password.encode()).hexdigest()
20     # Check if username exists and password matches
21     if username in user_db and user_db[username] == hashed_password:
22         return "Login successful!"
23     else:
24         return "Invalid username or password."
25
26 # Example usage
27 username_input = input("Enter username: ")
28 password_input = input("Enter password: ")
29 result = login_system(username_input, password_input)
30 print(result)
31
32
```

OUTPUT :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter username: chaitanya
Enter password: Chaitanya!88
Invalid username or password.
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:
Enter username: [ ]
```

FINAL DESCRIPTION :

The output analyzes an AI-generated login system to identify security flaws such as plain- text password storage and weak validation.

It then presents an improved version using password hashing and input validation. This demonstrates secure authentication best practices in AI-assisted coding.

Task Description #5 (Privacy in Data Logging)

Task: Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Expected Output:

- Identified privacy risks in logging.
- Improved version with minimal, anonymized, or masked

logging.

- Explanation of privacy-aware logging principles.

CODE :

```
ABHL.py > ...
31 # generate a python script that logs user activity including username ,IP address, timestamp of login and logout to a file named user_activity.log
32 import logging
33 from datetime import datetime
34 def log_user_activity(username, ip_address, action):
35     logging.basicConfig(filename='user_activity.log', level=logging.INFO,
36                         format='%(asctime)s - %(message)s')
37     timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
38     logging.info(f"User: {username}, IP: {ip_address}, Action: {action}, Timestamp: {timestamp}")
39 # Example usage with user input
40 username_input = input("Enter username: ")
41 ip_address_input = input("Enter IP address: ")
42 log_user_activity(username_input, ip_address_input, "login")
43 log_user_activity(username_input, ip_address_input, "logout")
44 # - Eliminates even numbers (except 2) and multiples of 3 (except 3) early.
45 # - The main optimization is iterating only up to the square root of n (i * i <= n).
46 #   If a number n has a divisor greater than its square root, it must also have a divisor
47 #   smaller than its square root. Therefore, we only need to check for divisors up
48 #   - After checking for 2 and 3, all primes greater than 3 can be expressed in the form 6k ± 1.
49 #   The loop increments `i` by 6 and checks `i` and `i`
50 #   effectively skipping 2/3 of the numbers that are not multiples of 2 or 3.
51 #   In the worst-case scenario, it performs divisions approximately up to sqrt(n).
52 #   The time complexity is roughly O(sqrt(n)).
53 #   The optimized version significantly improves performance, especially for large numbers,
54 #   because the square root of n grows much slower than n. For example, if n = 1,000,000,
55 #   the basic method might perform nearly a million checks, while the optimized method
56
57
58
```

OUTPUT :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Invalid username format. Use 3-20 alphanumeric characters or underscores.
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe

Enter username: Chaitanya@_123
Enter IP address: 162791628
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

FINAL DESCRIPTION :

- Eliminates even numbers (except 2) and multiples of 3 (except 3) early.
- The main optimization is iterating only up to the square root of n ($i * i \leq n$).
If a number n has a divisor greater than its square root, it must also have a divisor smaller than its square root. Therefore, we only need to check for divisors up
- After checking for 2 and 3, all primes greater than 3 can be expressed in the form $6k \pm 1$.

The loop increments `i` by 6 and checks `i` and `i`

effectively skipping 2/3 of the numbers that are not multiples of 2 or 3.

In the worst-case scenario, it performs divisions approximately up to $\sqrt{n}$.

The time complexity is roughly $O(\sqrt{n})$.

The optimized version significantly improves performance, especially for large numbers,

because the square root of n grows much slower than n. For example, if $n = 1,000,000$, the basic method might perform nearly a million checks, while the optimized method

